Initial evaluation: EuroGlobe Bank specification

Authors: Óscar Marín Castro, Víctor Jesús Castañeda Morales, Jaime de Luiz García, Martín Rodríguez Ramos, Pablo Polledo Blanco

Evaluated by: Fantastic 5 (implementing team)

Score overview

#	Criterion (from the assignment)	Score	One-line verdict
1	Intro / overview / audience	9/10	Clear product, problem and audience
2	Feature list / business requirements	6/10	Good MoSCoW list; only 2 user stories for 7 Musts
3	Business flows	7/10	All four core flows are diagrammed; key formulas are missing
4	Non-functional requirements	7/10	Measurable, but later contradicted
5	High-level architecture (package + class diagrams)	4/10	No package diagram, no system class diagram
6	UI/UX wireframes and transitions	7/10	Detailed and traceable; several screens are missing
7	Data model	3/10	Incomplete and contradicts the SQL file
8	API contract	5/10	Covers about half of the screens and use cases
9	Standalone / implementable without the authors	5/10	Implementable only after many assumptions by us
	Overall	**~6/10**	

1. Intro / overview / audience (9/10)

Good

- The product is described as a new product for the market..
- Four concrete pain points, each one tied to a module: mortgages, leasing, vaults and micro-savings.

- Three audience segments: individuals (including Erasmus students), SMEs, and groups.

Weak

- The SME segment promises "treasury management" and "multi-user transactional authorization", but no feature, flow or screen designs them.

2. Feature list / business requirements (6/10)

Good

- A clear MoSCoW split: 7 Must, 1 Should, 2 Could and 3 Won't items.
- The two user stories that exist have concrete acceptance criteria: the PENDING_REVIEW state, a minimum of 3 vault members, and a 24 h expiry..

Weak

- Only **2 user stories cover 7 Must requirements**. Registration, transfers, leasing, round-ups and cards have no acceptance criteria.
- Must #2 asks for "multi-currency accounts and SEPA Instant", but the API says EUR only, and no FX rules or SEPA behaviour are defined.
- Must #4 asks for "automated monthly billing", and Must #3 implies monthly repayments, but no process, scheduler or flow describes either.

3. Business flows (7/10)

Good

- Sequence diagrams for all four core flows, including the alternative and failure branches: DTI > 35 % rejection, a failed leasing check, and vault expiry.
- The business rules are numeric: DTI ≤ 35 %, LTV ≤ 85 %, 15 % minimum down payment, 2-of-3 quorum, 24 h window, round-up = ceil(x) − x.

Weak

- **There is no formula for mortgage payments.** The document gives no annuity formula, no fixed rate, and no Euribor value or spread. The example numbers disagree: for €200,000 over 300 months, 4.15 % gives about **€1,072/month**, but SCR-08 and SCR-10 show **€942**, which is less than the 3.65 % fixed option (€1,018). The API example (€1,043.17) matches neither.

- **There is no formula for the leasing monthly fee**, so €429/month (SCR-12) cannot be reproduced.
- Who approves a mortgage? Fig. 1 shows automatic, synchronous approval, but Story 1 implies a review state. No bank-employee or admin actor exists. The document also does not say what moves a mortgage into Delinquent.
- There are no flows for registration/KYC, transfers or card payments.

4. Non-functional requirements (7/10)

Good

- Every NFR has a number: p95 < 200 ms, 500 TPS, 10,000 concurrent users, 99.9 % uptime, RPO 15 min, RTO 4 h, TLS 1.3, AES-256.
- Fault isolation for investments is stated, and the event-driven design supports it.

Weak

- The "immutable audit log" has no entity in the data model.
- Throughput and availability targets are given with no deployment or infrastructure context, so we cannot verify them in this project.

5. High-level architecture (4/10)

Good

- Fig. 5 names the modules, and section 5 describes each module's responsibility.
- The design patterns are well explained. Each one has a reason and a small class diagram, and the mortgage state diagram (Fig. 8) is clear.

Weak

- **No package diagram and no system class diagram.** The assignment explicitly asks for both. The only class diagrams are isolated pattern examples.
- **No technology stack** (language, framework, database, frontend, event bus). The SQL file implies MySQL, but the document never says so.
- It is unclear whether "InvestmentMicroservice" is a separate deployable service or a module of a monolith.
- Some responsibilities are odd or missing. Card issuance sits in the IdentityModule, and no module owns card freezing or card payments.
- Traceability assigns Chain of Responsibility to transfers, Facade to registration, and State to cards, but section 6 describes none of these, and there is no card state diagram.

- The Composite pattern says that a GroupAccount "contains member accounts", while the vault has its own balance (SCR-17, SQL vaults.balance). The document does not say which model is meant.

6. UI/UX wireframes and transitions (7/10)

Good

- Every screen has an ID (SCR-01A to SCR-24). There is a global navigation diagram (Fig. 16) and a transition table per module that maps each button to a data binding, an action and a target screen. This is the strongest part of the document for implementers.
- The screens are consistent with the business rules (the DTI display, the 2-of-3 quorum, the expiry timer, the round-up receipt).

Weak

- Some screens are in the navigation diagram but have no wireframe: SCR-03 (Login) and SCR-07B (mortgage rejection). SCR-04 is never mentioned.
- Some use cases have no screen at all: **transfers** (the "+ New Transfer" button on SCR-05 has no target), card issuance, vault deposits, and vault invitation acceptance.
- No error or validation states, and no logout or profile screens.
- The wireframes bind to fields that are not in the data model: User.two_factor_secret, Amortization(applied_to), WithdrawalApproval(decision), Card.status = FROZEN.
- The SCR-01C button has two target screens (SCR-02 and SCR-05) with no rule for when each applies.

7. Data model (3/10)

The assignment asks for "a robust and detailed schema for **all** business entities".

Good

- Figure 15 uses compact table diagrams with data types, as the assignment requests.
- Money is stored as DECIMAL(19,2).

Weak

- **Figure 15 has only 6 entities.** It is missing Card, Transaction, Vault, VaultMember, WithdrawalApproval, Vehicle, InsuranceOption, KycRecord and AuditLog. User has no email, password or name, yet registration sends all three.
- **`euroglobe.sql` contradicts Figure 15:**
  - Different tables: the SQL has vaults and cards, but no withdrawal requests and no investments.
  - Different keys: INT instead of UUID.
  - Different relationships: Mortgage links to user_id instead of account_id.
  - Different account types: CHECKING/SAVINGS instead of individual/group.
  - No currency column.
- In the SQL, vault_approvals is unique on (vault_id, user_id) and has no withdrawal reference, so a member can approve only once per vault, ever.
- The traceability matrix names 13 entities that appear in neither model.
- **State values are never listed, and they disagree across sections:**
  - Mortgage: PENDING_REVIEW / UnderReview / pending_review / PENDING, and the SQL has no Delinquent.
  - Card: FROZEN vs BLOCKED.
  - Leasing: draft vs PENDING.

8. API contract (5/10)

Good

- Base path, JSON format, money format and status codes are defined.
- Request and response examples are given for every listed endpoint.
- One well-defined event (PaymentCompleted) with a clear rule.

Weak

- Section 9.2 says "12 endpoints", but the table lists 14. Many use cases have no endpoint:
  - card payments (needed for the whole round-up flow)
  - card freeze and PIN change
  - vehicle catalogue
  - investment portfolio (referenced in section 10 but not defined)
  - dashboard / account list
  - vault invitations and deposits
  - listing withdrawals
  - mortgage status
  - KYC document upload
  - 2FA setup
- POST /mortgages has no income or debt fields, so the required DTI check cannot be computed. rate_type is sent both at creation and in /rate-decision.

- /amortize has no field for "reduce term" vs "reduce payment", although SCR-10 offers that choice.
- No error response body is defined.
- No token details (header name, expiry).
- The event has no event_id.
- External services are not specified: KYC, credit bureau, Euribor, insurance, dealer, SEPA and asset prices. The document neither defines them nor says to mock them.

9. Standalone and implementable (5/10)

The document is well organised and clearly had an "implementer's hat" review (section 11.1 answers real edge cases). Even so, an implementing team **must decide on its own** about:

- the data model
- state names
- the financial formulas
- about 10 missing endpoints
- the security scope
- the technology stack